

UNIVERSIDADE FEDERAL DA FRONTEIRA SUL CAMPUS CHAPECÓ

CURSO: CIÊNCIA DA COMPUTAÇÃO

DISCIPLINA: ORGANIZAÇÃO DE COMPUTADORES

PROFESSOR: RICARDO PARIZOTTO

**RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DE BUSCA BINÁRIA COM
ALOCÇÃO DINÂMICA EM RISC-V**

AUTORES: ARTHUR KOLANKIEWICZ e GABRIEL FARINA

SUMÁRIO

- 1. Lógica do Algoritmo**
- 2. Estratégia de Alocação Dinâmica e Organização de Funções**
- 3. Desafios e Soluções (Erros de Desenvolvimento)**
 - 3.1 Deslocamento de Memória Incorreto (Busca Binária)**
 - 3.2 Loop Infinito por Convergência Inválida (Busca Binária)**
 - 3.3 Perda do Endereço Base no Heap (Alocação Dinâmica)**
- 4. Simulação e Evidências**
 - 4.1 Caso de Teste 1: Valor Ausente**
 - 4.2 Caso de Teste 2: Valor Presente**
- 5. Guia de Simulação e Restrições Técnicas**
- 6. Link para github com informações de execução**
- 7. Uso de IAs**

1. Lógica do Algoritmo

O projeto consiste na implementação do algoritmo de Busca Binária, pois se trata de um dos melhores algoritmos de busca.

- **Algoritmo:** A função `b_search` recebe o endereço base do array, o índice inicial (`low`), o índice final (`high`) e o valor alvo. A cada iteração, o ponto médio (`pivot`) é calculado através da soma dos índices seguida por um deslocamento aritmético para a direita (`srai`), equivalente à divisão por 2.
- **Comparação:** O valor no endereço calculado (`base + pivot * 4`) é comparado ao alvo. Dependendo do resultado, o espaço de busca é reduzido pela metade, ajustando-se os limites nos registradores `s0` (`low`) e `s1` (`high`) até que o elemento seja encontrado ou o intervalo se torne inválido.

2. Estratégia de Alocação Dinâmica e Organização de Funções

A organização do código foi dividida em um bloco principal para preparação de dados e uma função modularizada para a lógica de busca.

- **Alocação Dinâmica (Heap):** O programa utiliza a chamada de sistema `ecall 9` (`sbrk`) para alocar memória dinamicamente no heap. O tamanho necessário é calculado em tempo de execução multiplicando o valor na variável `tamanho` por 4.
- **Gerenciamento de Dados:** Após a alocação, os dados definidos na seção `.data` são copiados para o novo endereço no heap.
- **Convenção de Chamada:** A função `b_search` segue a convenção de chamadas do RISC-V, usando `Stack Pointer` que é essencial para salvar o endereço de retorno (`ra`) e os registradores `low` e `high`, permitindo que a função seja recursiva.

3. Desafios e Soluções (Erros de Desenvolvimento)

Durante o desenvolvimento, foram enfrentados desafios técnicos os quais podemos citar:

1. Deslocamento de Memória Incorreto (Busca Binária)

Erro: Inicialmente, o código somava o índice médio (pivot) diretamente ao endereço base sem multiplicá-lo por 4. Como o RISC-V endereça por byte e cada palavra possui 4 bytes, a busca acessava endereços desalinhados.

Solução: Inserção da instrução `slli t1, t0, 2` para converter o índice em deslocamento de bytes antes de acessar a memória.

2. Loop Infinito por Convergência Inválida (Busca Binária)

Erro: Ao atualizar os limites de busca, o código apenas movia o índice médio para os registradores de limite (low ou high). Isso causava um loop infinito quando o alvo não estava no array, pois os índices paravam de convergir.

Solução: Ajuste para que os novos limites sejam sempre `pivot + 1` ou `pivot - 1`, garantindo a redução do intervalo a cada iteração.

3. Perda do Endereço Base no Heap (Alocação Dinâmica)

Erro: Durante o processo de cópia dos dados da seção `.data` para o Heap, o registrador que armazenava o endereço inicial (retornado pelo `ecall 9`) era incrementado, fazendo com que a função de busca recebesse o ponteiro para o final do array, e não para o início.

Solução: Utilização de um registrador temporário para realizar o incremento durante a cópia, preservando o endereço base original para ser passado à função `b_search`.

4. Simulação e evidências

Para validar a implementação da busca binária, foi realizada uma execução com o seguinte conjunto de dados:

Array:

[2, 5, 8, 9, 10, 11, 12, 13, 14, 16, 23, 38, 56, 72, 91]

Valor alvo:

39

Saída esperada:

-1 (O valor 39 não está presente no array)

Durante a execução, os registradores indicam corretamente a operação de saída:

- **a0 (código da syscall):** 1 (indicando que a operação é de impressão de um valor inteiro, no entanto é sobrescrita por 10 para encerramento do programa)

- **a1 (valor a ser impresso): -1** (indicando que o valor alvo não foi encontrado)

Segue print do Venus, compilador utilizado para a implementação.

The screenshot displays the Venus MIPS simulator interface. The main window shows assembly code with columns for PC, Machine Code, Basic Code, and Original Code. The code includes instructions like `lui t0, tamanho`, `lw s1, 0(t0)`, `slli a1, s1, 2`, `li a0, 9`, `ecall`, `mv s0, a0`, `lui t0, dados`, `li t1, 0`, `beq s0, s1, run`, `lw t2, 0(t0)`, `sw t2, 0(a0)`, and `addi t0, t0, 4`. The right panel shows the register file with values for `zero` through `a7`. The `a1` register contains the value `-1`. The bottom console shows the output `-1`.

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	<code>lui pc x5 65536</code>	<code>la t0, tamanho</code>
0x4	0x04028293	<code>addi x5 x5 64</code>	<code>la t0, tamanho</code>
0x8	0x0002A483	<code>lw x9 0(x5)</code>	<code>lw s1, 0(t0)</code>
0xc	0x00249593	<code>slli x11 x9 2</code>	<code>slli a1, s1, 2</code>
0x10	0x00900513	<code>addi x10 x9 9</code>	<code>li a0, 9</code>
0x14	0x00000073	<code>ecall</code>	<code>ecall</code>
0x18	0x00050413	<code>addi x8 x10 0</code>	<code>mv s0, a0</code>
0x1c	0x10000297	<code>lui pc x5 65536</code>	<code>la t0, dados</code>
0x20	0xFE428293	<code>addi x5 x5 -28</code>	<code>la t0, dados</code>
0x24	0x00000313	<code>addi x6 x0 0</code>	<code>li t1, 0</code>
0x28	0x00930E53	<code>beq x6 x9 28</code>	<code>beq t1, s1, run</code>
0x2c	0x0002A383	<code>lw x7 0(x5)</code>	<code>lw t2, 0(t0)</code>
0x30	0x00752023	<code>sw x7 0(x10)</code>	<code>sw t2, 0(a0)</code>
0x34	0x00428293	<code>addi x5 x5 4</code>	<code>addi t0, t0, 4</code>

Register	Value
zero	0
ra (x1)	96
sp (x2)	2147483612
gp (x3)	268435456
tp (x4)	0
t0 (x5)	12
t1 (x6)	268408272
t2 (x7)	96
s0 (x8)	268408224
s1 (x9)	15
a0 (x10)	10
a1 (x11)	-1
a2 (x12)	14
a3 (x13)	39
a4 (x14)	0
a5 (x15)	0
a6 (x16)	0
a7 (x17)	0

Display Settings: Decimal

Saída no console:

-1

Para a demonstração de um caso de sucesso, utilizamos o mesmo array do exemplo anterior, alterando o valor alvo para 38. Espera-se que o algoritmo retorne com sucesso a posição do array onde o valor 38 está localizado. Conforme o array apresentado, a saída correta deve ser o valor 11, uma vez que essa é a posição em que o valor 38 se encontra.

Observe que, no registrador a1, está contido o valor da saída.

5. Guia de Simulação e Restrições Técnicas

Acessar <https://venus.cs61c.org/>.

Carregar o código fonte na aba Editor.

Aba Simulator -> clicar em Assemble & Simulate from Editor.

Rodar Step by Step, passo a passo;

ou Run, podendo visualizar a saída imediatamente.

Se desejar realizar simulações com outros exemplos de array, devem ser seguidas as condições abaixo.

- O array definido na seção .data deve obrigatoriamente estar ordenado de forma crescente. A lógica de "descarte de metades" baseia-se na transitividade da ordem; se o array estiver desordenado, o ponteiro de pivô não poderá determinar com segurança se o alvo está à esquerda ou à direita, levando a resultados inconsistentes ou falsos negativos. O valor de alvo também pode ser personalizado, permitindo o teste com diferentes valores a serem buscados.

6. Link do Github:

<https://github.com/akolankiewicz/binary-search-with-riscv>

Neste link, estão contidas mais algumas instruções, de forma resumida e com explicações para execução. Possuí tanto uma versão em Inglês, quanto uma em português.

7. Uso de IAs

O uso de IAs como Gemini e ChatGPT foram utilizados para processo de debug ao encontrar erros e como auxílio de entendimento do funcionamento do algoritmo de busca binária para assim implementá-lo. Também foram utilizadas para o refinamento de alguns tópicos apresentados neste relatório.